

Lab 2

Write a basic program to implement Depth First Search Algorithm

Date: _____

Objective

- To implement the Depth First Search (DFS) algorithm using an adjacency list to explore nodes in a graph or tree structure.
- To find and display the path from a given start node to a destination node, if such a path exists, using DFS traversal.

Theory

Depth First Search (DFS) is an uninformed search algorithm used for traversing or searching tree or graph data structures. The algorithm starts at a chosen root node and explores as far as possible along each branch before backtracking. It uses a stack data structure (either explicitly or via recursion) to keep track of the nodes to be explored next.

In DFS, a node is pushed onto the stack when it is discovered, and it is marked as visited only when it is popped and processed. This depth-first exploration makes DFS memory efficient compared to BFS, although it does not guarantee the shortest path in an unweighted graph. DFS is commonly used in applications such as maze solving, topological sorting, and cycle detection.

Implementation

Problem Statement

Given a tree with root node A branching into B and C; B further branches into D and E, while C branches into F. Starting from the initial state A, the goal is to reach the final state F using Depth First Search, exploring each branch as deep as possible before backtracking.

Python Code

```
adj_list = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [], 'E': [], 'F': []
}

def initialize_status(adj_list):
    return {node: 1 for node in adj_list}

def dfs_search(start, dest, adj_list):
    stack = [start]
    final_path = []
    status = initialize_status(adj_list)
    status[start] = 2

    while True:
        current = stack.pop()
        final_path.append(current)
        status[current] = 3
        if current == dest:
            break
```

```

        for neighbor in adj_list[current]:
            if status[neighbor] == 1:
                stack.append(neighbor)
                status[neighbor] = 2
            if not stack:
                break
        return final_path

def display_adjacency_list(adj_list):
    print("Adjacency List:")
    for key, value in adj_list.items():
        print(key, "->", value)

def main():
    display_adjacency_list(adj_list)
    start = input("Enter start node: ").upper()
    destination = input("Enter destination node: ").upper()
    path = dfs_search(start, destination, adj_list)
    print(f"Path from {start} to {destination}:")
    print(" -> ".join(path))
    print("End")

if __name__ == "__main__":
    main()

```

Output

Sample expected output (attach your own terminal screenshot below this in your final submission):

```

Adjacency List:
A -> ['B', 'C']
B -> ['D', 'E']
C -> ['F']
D -> []
E -> []
F -> []
Enter start node: A
Enter destination node: F
Path from A to F:
A -> C -> F
End

```

[Insert Screenshot Here]

Conclusion

The Depth First Search algorithm was successfully implemented using a stack-based approach and an adjacency list representation of the graph. The program correctly traversed the graph from the initial state A and found a path to the goal state F, demonstrating the depth-first exploration strategy in practice.

Lab 3

Write a basic program to implement Breadth First Search Algorithm

Date: _____

Objective

- To simulate Breadth First Search (BFS) algorithm using a queue for traversing a graph.

Theory

Breadth First Search (BFS) is an uninformed search algorithm that explores a graph level by level, starting from a given root node. It visits all the neighbors of a node before moving on to the next level of nodes, using a queue (First-In-First-Out) data structure to keep track of the nodes to be explored.

A node is enqueued when it is discovered and marked as visited once it is dequeued and processed. Because BFS explores nodes level by level, it guarantees the shortest path in terms of number of edges for unweighted graphs. BFS is widely used in applications such as shortest path finding, web crawling, and network broadcast.

Implementation

Problem Statement

Given a tree with root node A branching into B and C; B further branches into D and E, while C branches into F. Starting from the initial state A, the goal is to reach the final state F using Breadth First Search, exploring the graph level by level.

Python Code

```
from collections import deque

adj_list = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [], 'E': [], 'F': []
}

def initialize_status(adj_list):
    return {node: 1 for node in adj_list}

def bfs_search(start, dest, adj_list):
    queue = deque()
    final_path = []
    status = initialize_status(adj_list)
    queue.append(start)
    status[start] = 2

    while queue:
        current = queue.popleft()
        final_path.append(current)
        status[current] = 3
        if current == dest:
            break
        for neighbor in adj_list[current]:
            if status[neighbor] == 1:
                queue.append(neighbor)
```

```

        status[neighbor] = 2
    return final_path

def display_adjacency_list(adj_list):
    print("Adjacency List:")
    for key, value in adj_list.items():
        print(key, "->", value)

def main():
    display_adjacency_list(adj_list)
    start = input("Enter start node: ").upper()
    destination = input("Enter destination node: ").upper()
    path = bfs_search(start, destination, adj_list)
    print(f"Path from {start} to {destination}:")
    print(" -> ".join(path))
    print("End")

if __name__ == "__main__":
    main()

```

Output

Sample expected output (attach your own terminal screenshot below this in your final submission):

```

Adjacency List:
A -> ['B', 'C']
B -> ['D', 'E']
C -> ['F']
D -> []
E -> []
F -> []
Enter start node: A
Enter destination node: F
Path from A to F:
A -> B -> C -> F
End

```

[Insert Screenshot Here]

Conclusion

The Breadth First Search algorithm was successfully implemented using a queue-based approach and an adjacency list representation of the graph. The program correctly traversed the graph level by level from the initial state A and found the shortest path to the goal state F.

Lab 4

Write a basic program to implement Greedy Best First Search and find the path for the given problem

Date: _____

Objective

- To implement the Greedy Best First Search algorithm using an adjacency list to explore nodes in a graph or tree structure.
- To find and display the path from a given start node to a destination node, if such a path exists, using Greedy Best First Search traversal.

Theory

Greedy Best First Search (GBFS) is an informed search algorithm that uses a heuristic function $h(n)$ to estimate the cost from the current node to the goal node. Instead of exploring nodes in a strict level-by-level or depth-first manner, GBFS always expands the node that appears to be closest to the goal according to the heuristic value, using a priority queue (min-heap).

Because it relies purely on the heuristic and ignores the actual path cost accumulated so far, GBFS is generally faster than uninformed search methods but does not guarantee an optimal (shortest) path. Its performance depends heavily on the quality of the heuristic function used.

Implementation

Problem Statement

Given the tree A-B-C-D-E-F with heuristic values $h(A)=5$, $h(B)=2$, $h(C)=3$, $h(D)=4$, $h(E)=1$, $h(F)=0$, where A is the initial state and F is the goal state, find a path from A to F using Greedy Best First Search, always expanding the node with the lowest heuristic value.

Python Code

```
import heapq

adj_list = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [], 'E': [], 'F': []
}

heuristic = {'A': 5, 'B': 2, 'C': 3, 'D': 4, 'E': 1, 'F': 0}

def gbfs(start, goal):
    visited = set()
    pq = [(heuristic[start], start)]
    path = []
    while pq:
        h, node = heapq.heappop(pq)
        if node in visited:
            continue
        path.append(node)
        visited.add(node)
        if node == goal:
```

```

        break
    for neighbor in adj_list[node]:
        if neighbor not in visited:
            heapq.heappush(pq, (heuristic[neighbor], neighbor))
    return path

def display_adjacency_list(adj_list):
    print("Adjacency List:")
    for node, neighbors in adj_list.items():
        print(node, "->", neighbors)

def main():
    display_adjacency_list(adj_list)
    start = input("Enter start node: ").upper()
    destination = input("Enter destination node: ").upper()
    path = gbfs(start, destination)
    print(f"Path from {start} to {destination}:")
    print(" -> ".join(path))
    print("End")

if __name__ == "__main__":
    main()

```

Output

Sample expected output (attach your own terminal screenshot below this in your final submission):

```

Adjacency List:
A -> ['B', 'C']
B -> ['D', 'E']
C -> ['F']
D -> []
E -> []
F -> []
Enter start node: A
Enter destination node: F
Path from A to F:
A -> C -> F
End

```

[Insert Screenshot Here]

Conclusion

The Greedy Best First Search algorithm was successfully implemented using a priority queue and heuristic values. The program expanded nodes based on their heuristic estimate to the goal and found a path from A to F, illustrating how heuristic-guided search can reach the goal faster than uninformed methods, though not always via the shortest route.

Lab 5

Write a basic program to implement A* Search and find the path for the given problem

Date: _____

Objective

- To implement the A* Search algorithm using an adjacency list to explore nodes in a graph or tree structure.
- To find and display the path from a given start node to a destination node using A* Search traversal.

Theory

A* Search is an informed search algorithm that combines the strengths of Uniform Cost Search and Greedy Best First Search. It evaluates nodes using the function $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost from the start node to the current node, and $h(n)$ is the heuristic estimate of the cost from the current node to the goal.

By considering both the accumulated path cost and the heuristic estimate, A* is able to find the optimal (least-cost) path to the goal, provided the heuristic function is admissible (never overestimates the true cost). It uses a priority queue to always expand the node with the lowest $f(n)$ value first, making it one of the most widely used algorithms in pathfinding and graph traversal problems.

Implementation

Problem Statement

Given the weighted graph A-B-C-D-E-F with edge costs and heuristic values $h(A)=7$, $h(B)=6$, $h(C)=2$, $h(D)=4$, $h(E)=1$, $h(F)=0$, where A is the initial state and F is the goal state, find the least-cost path from A to F using A* Search.

Python Code

```
import heapq

graph = {
    'A': [('B', 1), ('C', 4), ('D', 3)],
    'B': [('E', 2)],
    'C': [('E', 1), ('F', 5)],
    'D': [('F', 6)],
    'E': [('F', 1)],
    'F': []
}

heuristic = {'A': 7, 'B': 6, 'C': 2, 'D': 4, 'E': 1, 'F': 0}

def a_star_search(start, goal):
    open_set = [(heuristic[start], 0, start, [start])]
    visited = set()
    while open_set:
        f, g, current, path = heapq.heappop(open_set)
        if current == goal:
            return path
```

```

        if current in visited:
            continue
        visited.add(current)
        for neighbor, cost in graph[current]:
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbor]
                heapq.heappush(open_set, (new_f, new_g, neighbor, path + [neighbor]))
    return None

def main():
    print("Graph adjacency list with costs:")
    for node, neighbors in graph.items():
        print(node, "->", neighbors)

    start, goal = 'A', 'F'
    path = a_star_search(start, goal)
    if path:
        print("Shortest path from start to goal using A* Search:")
        print(" -> ".join(path))
    else:
        print("No path found from start to goal.")

if __name__ == "__main__":
    main()

```

Output

Sample expected output (attach your own terminal screenshot below this in your final submission):

```

Graph adjacency list with costs:
A -> [('B', 1), ('C', 4), ('D', 3)]
B -> [('E', 2)]
C -> [('E', 1), ('F', 5)]
D -> [('F', 6)]
E -> [('F', 1)]
F -> []
Shortest path from start to goal using A* Search:
A -> B -> E -> F

```

[Insert Screenshot Here]

Conclusion

The A* Search algorithm was successfully implemented using a priority queue combining path cost $g(n)$ and heuristic estimate $h(n)$. The program found the optimal least-cost path from A to F, confirming that A* guarantees optimality when the heuristic function is admissible.

Lab 6

Write a basic program to implement minmax algorithm from game playing theory for the given problem and find the optimal path

Date: _____

Objective

- To implement the minmax algorithm from game playing theory and solve the given problem using Python.

Theory

The Minimax algorithm is a decision-making algorithm used in two-player, zero-sum games such as Tic-Tac-Toe and Chess. It assumes that both players play optimally: the Maximizing player tries to achieve the highest possible score, while the Minimizing player tries to achieve the lowest possible score.

The algorithm works by recursively exploring the game tree down to the leaf (terminal) nodes, which hold the utility values of the game outcomes. It then backs up these values through the tree: at each Maximizer node it picks the maximum of the child values, and at each Minimizer node it picks the minimum. This backtracking continues until the root node yields the optimal value, representing the best achievable outcome assuming perfect play from both sides.

Implementation

Problem Statement

Given a game tree with root node A, having two children B and C, and each of B and C having two leaf nodes with utility values [3, 5, 2, 9] respectively, determine the optimal value at the root using the Minimax algorithm, assuming A is the maximizing player.

Python Code

```
def minimax(depth, node_index, is_maximizing, values, max_depth):
    if depth == max_depth:
        return values[node_index]

    if is_maximizing:
        left = minimax(depth + 1, node_index * 2, False, values, max_depth)
        right = minimax(depth + 1, node_index * 2 + 1, False, values, max_depth)
        return max(left, right)
    else:
        left = minimax(depth + 1, node_index * 2, True, values, max_depth)
        right = minimax(depth + 1, node_index * 2 + 1, True, values, max_depth)
        return min(left, right)

def main():
    values = [3, 5, 2, 9]
    max_depth = 2
    optimal_value = minimax(0, 0, True, values, max_depth)
    print("The optimal value is:", optimal_value)

if __name__ == "__main__":
    main()
```

Output

Sample expected output (attach your own terminal screenshot below this in your final submission):

```
The optimal value is: 3
```

[Insert Screenshot Here]

Conclusion

The Minimax algorithm was successfully implemented to evaluate a simple two-level game tree. The program correctly backed up the leaf node values through alternating maximizing and minimizing levels to determine the optimal value at the root, demonstrating how game-playing agents can select optimal moves under adversarial conditions.

Lab 7

Write a program to solve the following water jug problem using the approach of constraint satisfaction problem

Date: _____

Objective

- To implement the water jug problem solution using the constraint satisfaction problem approach.

Theory

A Constraint Satisfaction Problem (CSP) is defined by a set of variables, their possible domains, and a set of constraints that restrict the values the variables can simultaneously take. Solving a CSP involves finding an assignment of values to variables such that all constraints are satisfied.

The Water Jug Problem is a classic CSP: given two jugs of fixed capacities and no measuring marks, the goal is to measure out a target quantity of water using only fill, empty, and pour operations, while respecting the capacity constraints of each jug ($0 \leq a \leq \text{max_a}$ and $0 \leq b \leq \text{max_b}$). Each valid operation generates a new state, and a search strategy such as Breadth First Search can be used to explore the state space and find a sequence of operations that reaches the goal state.

Implementation

Problem Statement

Given Jug A with a capacity of 5 litres and Jug B with a capacity of 3 litres, both starting empty (0, 0), determine a sequence of fill, empty, and pour operations that results in Jug A containing exactly 1 litre of water, using a constraint satisfaction based BFS search over valid states.

Python Code

```
from collections import deque

max_a, max_b = 5, 3

def is_goal(state):
    return state[0] == 1

def get_next_states(state):
    a, b = state
    successors = []
    successors.append((max_a, b))          # Fill Jug A
    successors.append((a, max_b))          # Fill Jug B
    successors.append((0, b))              # Empty Jug A
    successors.append((a, 0))              # Empty Jug B
    pour = min(a, max_b - b)
    successors.append((a - pour, b + pour)) # Pour A -> B
    pour = min(b, max_a - a)
    successors.append((a + pour, b - pour)) # Pour B -> A
    return successors

def water_jug_bfs(start_state):
    visited = set()
    queue = deque([(start_state, [start_state])])
    while queue:
        current, path = queue.popleft()
```

```

        if is_goal(current):
            return path
        for next_state in get_next_states(current):
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [next_state]))
    return None

def main():
    solution = water_jug_bfs((0, 0))
    if solution:
        print("Step-by-step jug contents to goal state:")
        for step in solution:
            print(f"Jug A: {step[0]}L, Jug B: {step[1]}L")
    else:
        print("No solution found")

if __name__ == "__main__":
    main()

```

Output

Sample expected output (attach your own terminal screenshot below this in your final submission):

```

Step-by-step jug contents to goal state:
Jug A: 0L, Jug B: 0L
Jug A: 5L, Jug B: 0L
Jug A: 2L, Jug B: 3L
Jug A: 2L, Jug B: 0L
Jug A: 0L, Jug B: 2L
Jug A: 5L, Jug B: 2L
Jug A: 4L, Jug B: 3L
Jug A: 4L, Jug B: 0L
Jug A: 1L, Jug B: 3L

```

[Insert Screenshot Here]

Conclusion

The Water Jug Problem was successfully solved by modeling it as a constraint satisfaction problem and applying a Breadth First Search over valid states generated by fill, empty, and pour operations. The program correctly found a sequence of operations that resulted in the goal state where Jug A contained exactly 1 litre of water.